

-- { БАЗОВЫЙ УРОВЕНЬ - ОСНОВЫ PYTHON } --

Ethical Hacking Tutorial Group The Codeby

представляет



Основы программирования на

PYTHON



-- { для начинающих } --

Содержание

ВВЕДЕНИЕ	2
Установка Python.....	3
Альтернативная установка в Linux.....	4
Установка Python в Windows.....	5
Как запускать программы Python.....	6
Режимы интерпретатора Python.....	6
Online IDE	8
Поддержка python в текстовых редакторах	8
Установка PyCharm	9
Работа в IDE PyCharm.....	13
Основы синтаксиса Python	14
Переменные в Python.....	14
Строки и отступы	17
Комментарии в Python.....	19
Настройки IDE PyCharm.....	20
Установка ограничения максимальной длины строки	20
Отключение проверки орфографии	21
Домашнее задание:	22

Введение

Язык программирования **Python** был создан нидерландским разработчиком **Гвидо ван Россумом (Guido van Rossum)** в 1989 г.



Официальный логотип



Python – это интерпретируемый, объектно-ориентированный язык программирования высокого уровня с динамической типизацией, и автоматическим управлением памятью.

Официальный сайт <https://www.python.org/>

Язык Python существует в двух версиях 2 и 3. Вторая ветка устарела, и официально перестала поддерживаться с января 2020 года. Синтаксис второй и третьей версии отличается, поэтому программа, написанная на python2, и запущенная в python3, выдаст ошибки, и не сможет корректно выполниться.

Разработчики языка Python придерживаются определённой философии программирования, называемой «**The Zen of Python**». Вот некоторые тезисы:

- Красивое лучше, чем уродливое.
- Явное лучше, чем неявное.
- Простое лучше, чем сложное.
- Ошибки никогда не должны замалчиваться.

Язык пайтон широко используется в своих проектах такими компаниями как

- ✓ Google
- ✓ Dropbox
- ✓ Mozilla
- ✓ Facebook
- ✓ Yandex
- ✓ Red Hat
- ✓ Microsoft
- ✓ Instagram
- ✓ YouTube
- ✓ PayPal

и многими другими.

Установка Python

В операционных системах Linux чаще всего Python уже предустановлен. Проверить наличие Python можно, набрав в терминале `python -V`. Выведется версия, которая установлена по умолчанию. На самом деле в системе может быть уже установлено несколько версий. Увидеть все версии можно выполнив команду `ls /usr/bin/python*`

```
explorer@exp:~$ python -V
Python 2.7.16
explorer@exp:~$ ls /usr/bin/python*
/usr/bin/python
/usr/bin/python2
/usr/bin/python2.7
/usr/bin/python2.7-config
/usr/bin/python2.7-dbg
/usr/bin/python2.7-dbg-config
/usr/bin/python2-config
/usr/bin/python2-dbg
/usr/bin/python2-dbg-config
/usr/bin/python2-futurize
/usr/bin/python2-pasteurize
/usr/bin/python2-qr
/usr/bin/python2-wsdump
/usr/bin/python3
/usr/bin/python3.7
/usr/bin/python3.7-config
/usr/bin/python3.7-dbg
/usr/bin/python3.7-dbg-config
/usr/bin/python3.7dm
/usr/bin/python3.7dm-config
/usr/bin/python3.7m
/usr/bin/python3.7m-config
/usr/bin/python3.8
```

В ОС на базе Debian python устанавливается следующим образом:

обновляем список пакетов. Команда sudo запускает следующую за ней команду от имени супер пользователя, запрашивает пароль, тем самым защищая систему

`sudo apt update`

устанавливаем необходимые дополнительные пакеты.

`sudo apt -y install wget build-essential libncursesw5-dev libssl-dev libsqlite3-dev tk-dev libgdbm-dev libc6-dev libbz2-dev libffi-dev zlib1g-dev`

устанавливаем пакет python3

`sudo apt -y install python3`

Для удобства запуска python можно создать алиас (псевдоним/ярлык) команды в терминале

`sudo alias python=python3`

Чтобы этот псевдоним загружался каждый раз при запуске терминала, нужно добавить его в файл настроек терминала

```
sudo echo alias python=python3 >> ~/.zshrc
```

Альтернативная установка в Linux

Так же мы можем установить и любую другую версию Python из исходников. Для этого на официальном сайте переходим в [раздел](#) Downloads и там, в кнопке Download Python будет указана последняя актуальная версия для загрузки:



После нажатия на кнопку начнётся скачивание архива.

Мы так же можем найти нужную версию Python на [сервере](#), и скачать её прямо с ftp. Сделать это в терминале можно следующим образом:

```
# переходим в каталог для загрузки архива
cd /tmp

# скачиваем файл
wget https://www.python.org/ftp/python/3.11.4/Python-3.11.4.tgz

# распаковываем файл
tar xvf Python-3.11.4.tgz

# переходим в извлечённый каталог
cd Python-3.11.4

# запускаем скрипт для проверки и установки
./configure --enable-optimizations

# собираем всё
sudo make altinstall

# удаляем скачанный архив и переносим папку с исходниками в папку /opt/
cd ../ && sudo rm --rf Python-3.11.4.tgz && mv Python-3.11.4 /opt/
```

Нужно учитывать, что после подобной установки в системе будет сосуществовать несколько версий интерпретатора, каждая из которых вызывается своей командой:

```
explorer@exp: ~ $ python3.8
Python 3.8.3 (default, Jun 11 2020, 12:24:57)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()
explorer@exp: ~ $ python3.11
Python 3.11.4 (main, Jul 1 2023, 18:02:49) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> exit()
explorer@exp: ~ $
```

Чтобы команда `python` запускала именно версию 3.11, точно так же создаём алиас и добавляем его в файл настроек терминала `.zshrc`:

```
sudo echo alias python=python3.11 >> ~/.zshrc
```

Можно для краткости добавить и такой алиас

```
sudo echo alias py=python3.11 >> ~/.zshrc
```

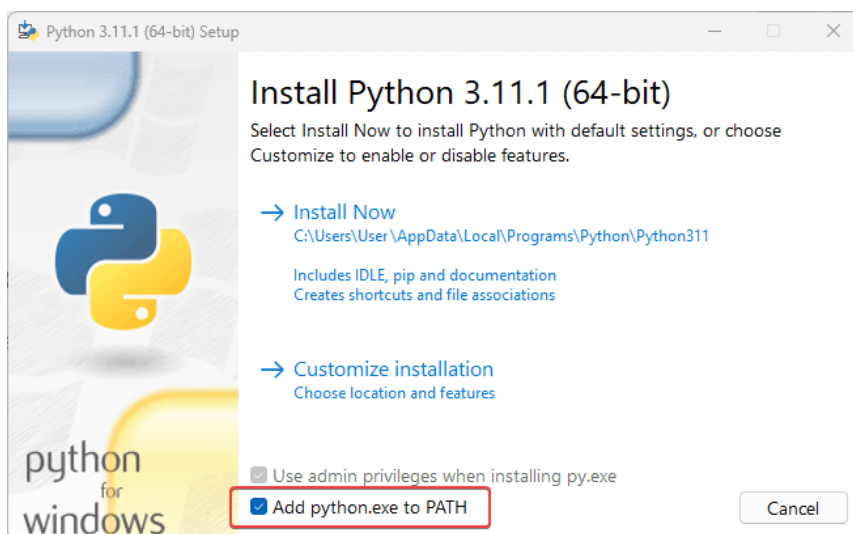
После добавления алиасов, нужно или перезапустить терминал, или актуализировать настройки командой

```
sudo source ~/.zshrc
```

Теперь для вызова версии 3.11 достаточно ввести всего 2 буквы - `py`.

Установка Python в Windows

Для установки Python в **Windows** мы так же переходим на официальную страницу, скачиваем exe файл, запускаем его и следуем инструкции. Не забудьте поставить галочку в поле Add Python to PATH



Как запускать программы Python

Программы, написанные на языке Python, чаще всего имеют расширение `.py`. Но имейте в виду, что любой текстовый файл, содержащий код на языке Python, можно попытаться выполнить, передав его имя первым аргументом программы `python`:

```
explorer@exp: ~ $ cat test_python
print('1 2 3')
print('Codeby')
explorer@exp: ~ $ python test_python
1 2 3
Codeby
```

В этом примере у нас есть файл `test_python`, как видите, у него отсутствует расширение. Утилитой `cat` мы вывели на консоль его содержимое, а команда `python test_python` смогла выполнить код, который там написан, выведя на экран 2 строки.

Режимы интерпретатора Python

Python может работать в двух режимах:

- интерактивный
- пакетный (скриптовый)

В интерактивный режим можно войти, набрав в терминале `python` или `python3`, или `python<установленная версия>`

```
explorer@exp: ~ $ python2
Python 2.7.18 (default, Mar 28 2022, 20:47:09)
[GCC 11.2.0] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>>
explorer@exp: ~ $ python
Python 3.8.3 (default, Jun 11 2020, 12:24:57)
[GCC 9.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
explorer@exp: ~ $ python3.11
Python 3.11.4 (main, Jul 1 2023, 18:02:49) [GCC 11.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

В этом режиме можно выполнять команды, например, вывести на печать строку, или воспользоваться как калькулятором для выполнения арифметических вычислений.

Наберите команду `print("Hello, World!")`, и нажмите ENTER. В ответ интерпретатор выведет Hello, World! А набрав `2+2`, получим сумму 4:

```
explorer@exp:~$ python3
Python 3.7.3 (default, Apr  3 2019, 05:39:12)
[GCC 8.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
>>> 2+2
4
>>> █
```

Основной режим использования – именно пакетный. Нужно создать пустой файл в любом текстовом редакторе, и сохранить его с расширением `py`, например `mainfile.py`

Также это можно сделать командами в терминале:

Перейдём в домашнюю директорию, создадим каталог `lesson` и файл `mainfile.py`

```
explorer@exp:~$ cd /home
explorer@exp:/home$ sudo mkdir lesson
[sudo] пароль для explorer:
explorer@exp:/home$ cd lesson
explorer@exp:/home/lesson$ sudo touch mainfile.py
```

В этом файле мы напишем тот же текст `print("Hello, World!")`, и сохраним. Таким образом, мы создали простейшую первую программу на Python.

Запуск программы в терминале осуществляется следующим образом:

`python /lesson/mainfile.py` или `python3 /lesson/mainfile.py`, либо сначала перейти в директорию с программой, и запустить уже не указывая путь `python3 mainfile.py`.

```
explorer@exp:/home/lesson$ python3 mainfile.py
Hello, World!
explorer@exp:/home/lesson$ cd
explorer@exp:~$ python3 /home/lesson/mainfile.py
Hello, World!
explorer@exp:~$ █
```

IDE (англ. Integrated Development Environment) - интегрированная среда разработки)

Итак, как мы выяснили, для создания программы Python, достаточно написать код в любом текстовом редакторе, и сохранить его. Однако существуют специально разработанные программы, в которых есть множество удобных и нужных функций для написания, редактирования и запуска кода, они называются средами разработки – IDE. Они поддерживают и подсветку синтаксиса, анализ ошибок в коде, пошаговый запуск кода, различные темы оформления и т.д.

По ссылкам ниже можно ознакомиться с доступными средами разработки. У каждой из них есть свои плюсы и минусы. Поэтому можно попробовать разные варианты и остановиться на той, которая больше нравится.

[10 Лучших IDE для Python](#)

[Python IDE — какой редактор кода выбрать?](#)

Стоит отметить, что одной из самых функциональных сред разработки для Python, является IDE PyCharm от компании JetBrains. Именно её мы и рассмотрим подробнее ниже.

Online IDE

Наряду с программами, устанавливаемыми на устройство пользователя, мы можем использовать и «виртуальные» среды разработки. Их функционал куда скромнее, но для выполнения простого кода, тестирования части кода или просто для тренировки их возможностей более чем достаточно. Другим важным их преимуществом является возможность запуска в браузере на любых устройствах – начиная от смартфона, планшета, заканчивая ноутбуком или моноблоком.

Среди наиболее популярных можно выделить

[repl.it](#)
[programiz](#)
[w3schools](#)

Поддержка python в текстовых редакторах

Выполнять код на языке python можно и с помощью дополнительных плагинов и аддонов в таких текстовых редакторах, как Sublime Text, notepad++, Visual Studio Code и других. Мы все же рекомендуем установить PyCharm. Это связано с тем, что все примеры кода в видеоуроках будут демонстрироваться именно в PyCharm, поэтому вам будет проще ориентироваться и следовать инструкциям.

Установка PyCharm

На данный момент существуют 2 версии PyCharm – Professional Edition, Community Edition.

Professional – платная версия с более широким функционалом

Community – бесплатная версия с меньшими возможностями

Для наших нужд версия Community подойдёт лучше всего. Для её установки переходим на [официальную](#) страницу компании JetBrains, эта компания как раз и выпускает IDE. Прокручиваем страницу вниз до таблицы сравнения и под колонкой **PyCharm Community Edition**, нажимаем на ▼ рядом с .exe и выбираем tar.gz (Linux).

	PyCharm Professional Edition	PyCharm Community Edition
Функциональный редактор Python	✓	✓
Инструмент запуска тестов и графический отладчик	✓	✓
Навигация по коду и рефакторинг	✓	✓
Инспекции кода	✓	✓
Поддержка систем контроля версий	✓	✓
Инструменты для научных вычислений	✓	
Веб-разработка	✓	
Веб-фреймворки Python	✓	
Python-профилировщик	✓	
Возможности удаленной разработки	✓	
Поддержка баз данных и SQL	✓	

Скачать .exe ▼

Бесплатная пробная версия

Скачать .exe ▲

.exe (Windows)

.exe (Windows ARM64)

.dmg (macOS Intel)

.dmg (macOS Apple Silicon)

.tar.gz (Linux)

... (Linux ARM64)

PyCharm для обучения

Хотите научиться программировать на Python? Или, может быть, обучить других?

У нас есть для вас идеальный инструмент.

[ПОПРОБОВАТЬ](#)

Скачивание начнётся автоматически. После его окончания переходим в каталог загрузки, распаковываем архив в директорию /opt/, переходим в эту директорию, дальше поддиректорию bin – там хранятся исполняемые файлы нашей IDE и запускаем её через файл ./pycharm.sh:

распаковываем архив в папку /opt/ и удалим скачанный его

```
sudo tar xf PyCharm-community-2023.1.3.tar.gz -C /opt/ && rm PyCharm-community-2023.1.3.tar.gz
```

переходим в директорию opt
`cd /opt/`

убедимся, что архив распаковался

```
ls
```

перейдём в каталог bin

```
cd PyCharm-community-2023.1.3/bin
```

и запустим скрипт

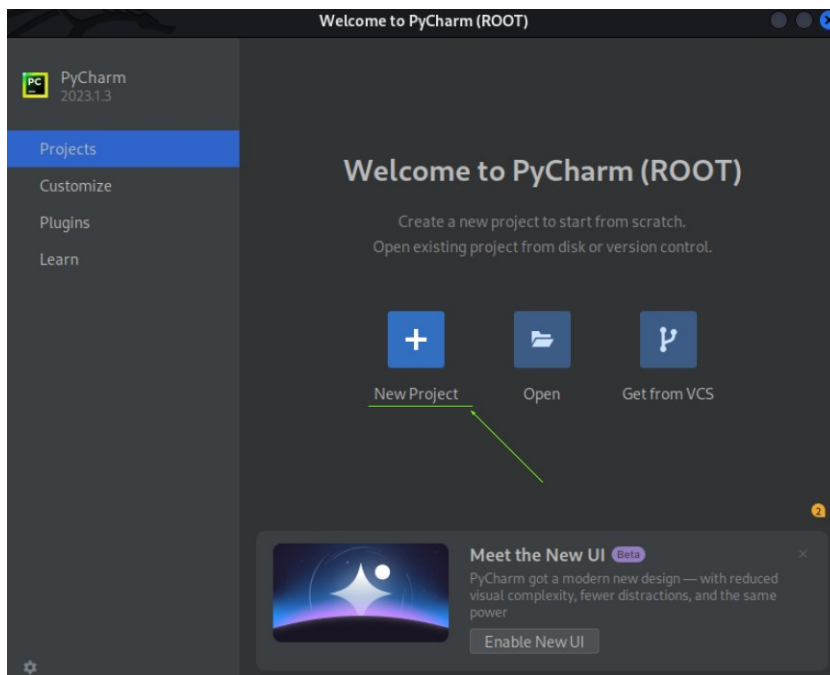
```
./PyCharm.sh
```

```
explorer@exp: ~/Downloads $ /bin/ls
pycharm-community-2023.1.3.tar.gz
explorer@exp: ~/Downloads $ file pycharm-community-2023.1.3.tar.gz
pycharm-community-2023.1.3.tar.gz: gzip compressed data, original size modulo 2^32 1776807936
explorer@exp: ~/Downloads $ tar xf pycharm-community-2023.1.3.tar.gz -C /opt/
explorer@exp: ~/Downloads $ cd /opt/
explorer@exp: /opt $ /bin/ls
pycharm-community-2023.1.3
explorer@exp: /opt $ cd pycharm-community-2023.1.3/bin
explorer@exp: /opt/pycharm-community-2023.1.3/bin $ ./pycharm.sh
```

Нас встречает приветственный логотип:



...и окно приглашения



Жмём на кнопку + New Project

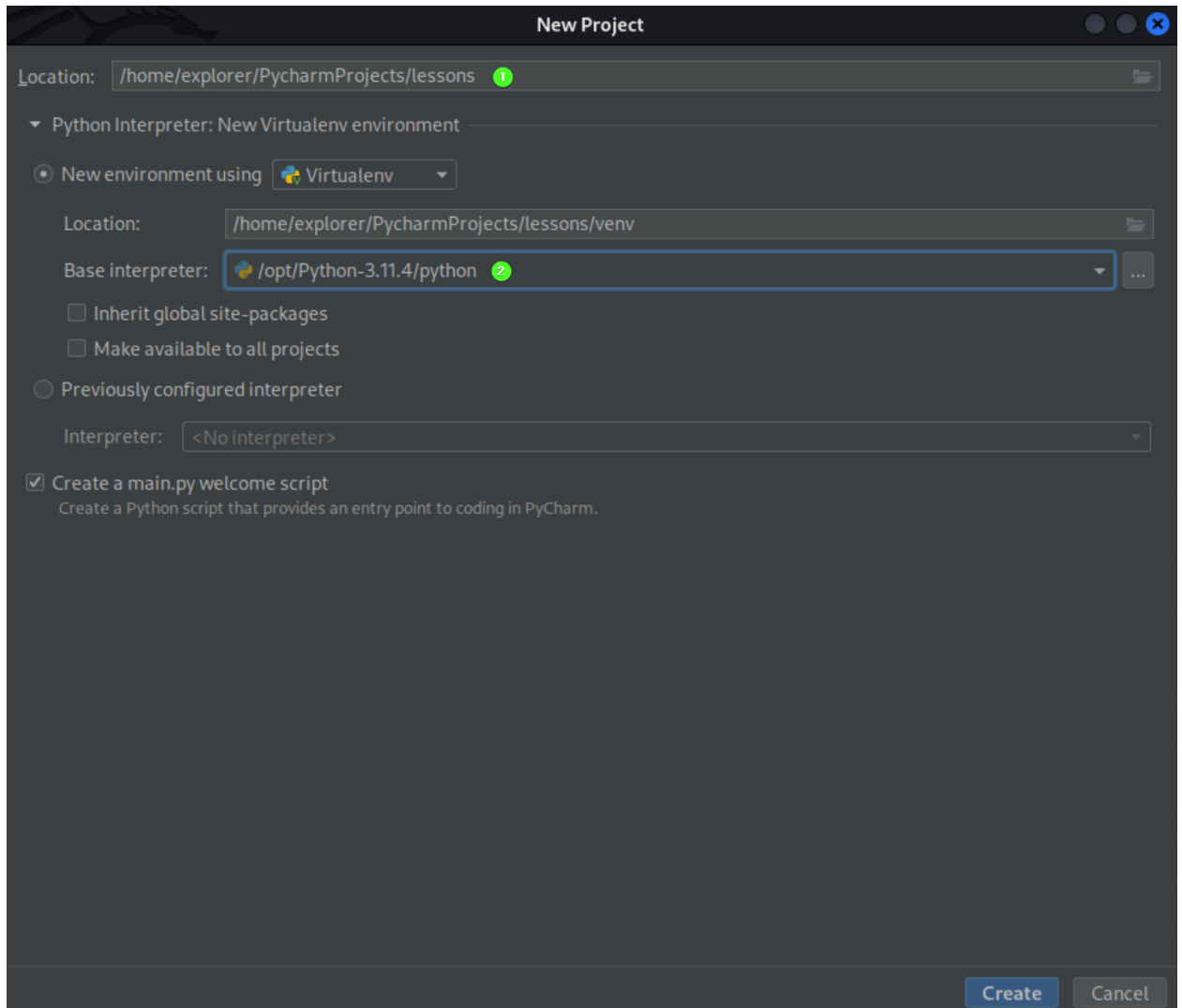
Перед нами появляется окно настроек интерпретатора [python](#). Здесь мы можем указать основные настройки нашего проекта. Проект – это группа файлов и папок, относящихся к одной программе. При написании объемного или сложного кода считается хорошим тоном разбивать код на блоки, а большие блоки переносить в файлы. Таким образом, мы осуществляем декомпозицию задачи – разбиваем одну большую задачу на несколько мелких – об этом мы ещё поговорим в следующих уроках.

Сейчас же мы указываем путь к нашему проекту (1), проверяем настройки виртуального окружения, версии python и пути, по которому расположен файл с нашим интерпретатором (2).

Виртуальное окружение – Virtual environment - представляет собой инструмент, который организует в себе необходимые версии пакетов требуемых только для нашей программы. То есть мы устанавливаем эти пакеты не в основной папке [python](#), не мешаем другим программам, а работаем только в своём окружении – поэтому и называется виртуальное окружение.

Удобство такого подхода в том, что вы можете создать сколь угодно много подобных проектов, ссылаясь на одно и то же виртуальное окружение, если в ваших программах требуются одни и те же модули и пакеты.

Далее проверяем путь к интерпретатору [python](#) – или выбираем нужную версию из выпадающего списка, или указываем путь самостоятельно – выше мы переносили папку с исходниками в директорию `/opt/Python-3.11.4/`, значит, указываем её и путь к файлу python - `/opt/Python-3.11.4/python`



Жмём Create, и наш первый проект PyCharm запущен!

Чтобы нам каждый раз не прописывать путь для запуска PyCharm в терминале, создадим алиас командой

```
echo alias PyCharm='/opt/ PyCharm-community-2023.1.3/bin/PyCharm.sh'
```

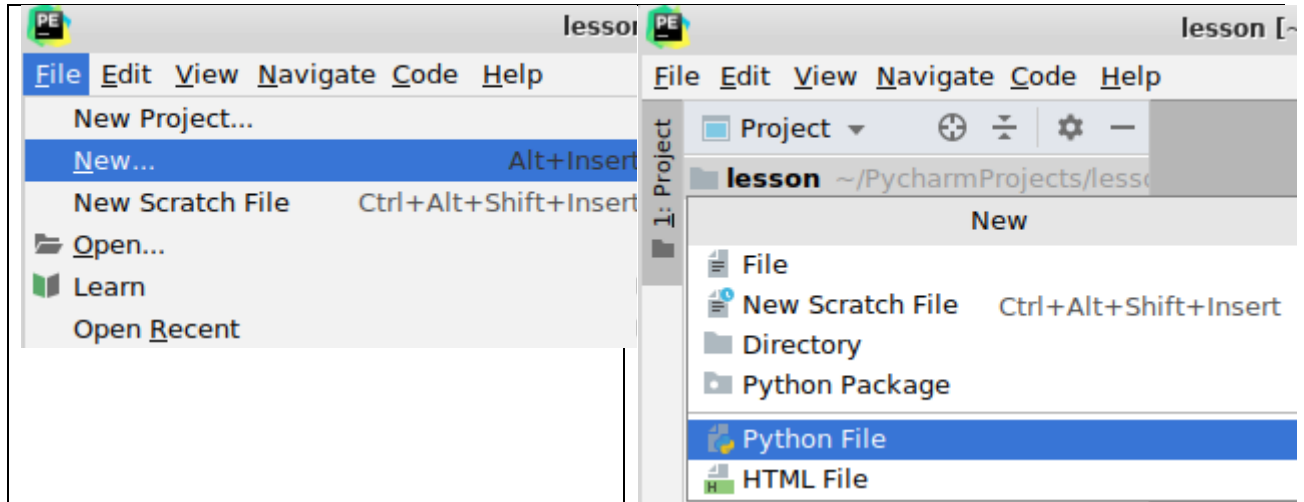
```
# перезагружаем файл .bashrc
```

```
source ~/.zshrc
```

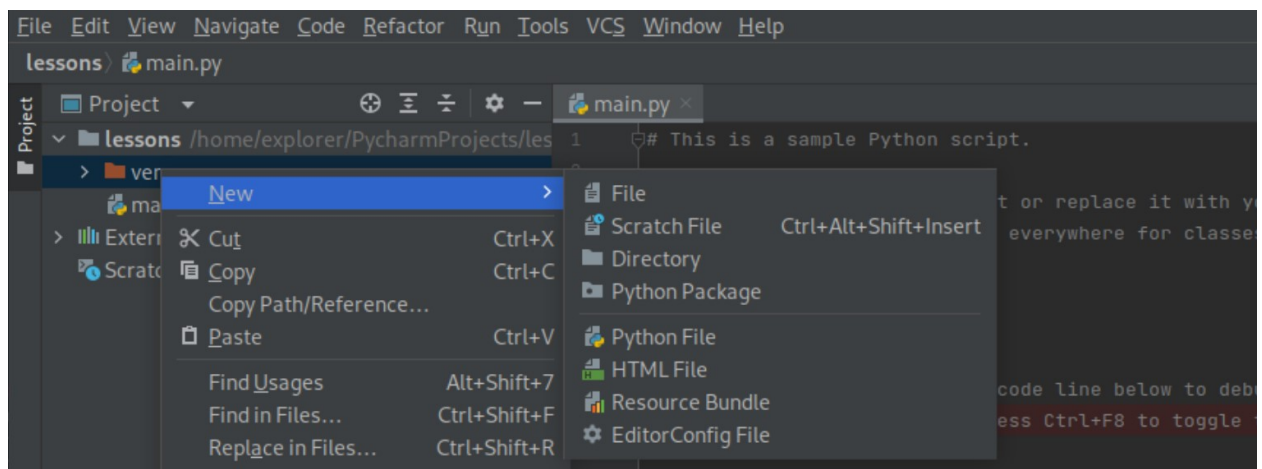
Теперь для запуска IDE, нужно лишь набрать в терминале команду **PyCharm** и нажать Enter для запуска IDE.

Работа в IDE PyCharm

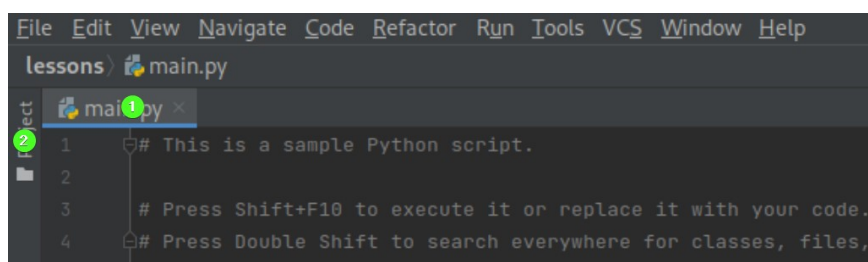
При создании проекта PyCharm сама создала нам «приветственный» файл, где показала основные возможности и горячие клавиши. По умолчанию этот файл имеет имя `main.py` – старайтесь и в своих проектах главный файл именовать точно так же. Можно использовать этот файл или создать свой - делается это очень просто **File > New > Python File > Name** пусть будет файл `test`.



Другим вариантом создания нового файла в проекте будет клик правой кнопкой по папке проекта и в выпадающем меню так же выбор **New > Python File**:



Если вдруг вы случайно закрыли окно с содержимым проекта, просто дважды кликните на имя файла – в нашем случае `main.py` - на вкладке файлов (1) или на кнопку Project слева (2)



Основы синтаксиса Python

Прежде чем мы будем писать программы, необходимо ознакомиться с базовыми понятиями языка пайтон.

Переменные в Python

Переменная в Python – это имя, используемое для идентификации объекта, функции, класса, модуля или другого объекта. Переменная может начинаться с любой буквы английского алфавита в верхнем или нижнем регистре, или со знака нижнего подчеркивания. Сразу приучайте себя к правильному именованию переменных – используйте словарные слова английского языка или их общепринятые сокращения.

Python не разрешает такие символы как @, \$ и % в пределах имени переменной. Так же Python чувствителен к регистру. Таким образом, Codeby и codeby являются двумя различными идентификаторами в Python.

Основные соглашения для переменных в **python**:

➡ Для именования обычных переменных используется **змеиная нотация (snake_case)**. Каждое слово, а так же цифра, в переменной пишется с маленькой буквы, отделяется от другого нижним подчёркиванием, слова используются только из английского языка. Использование транслитерации с других языков крайне не приветствуется.

my_var, var_1, new_object, target_color, user_3

➡ Для именования классов используется **верблюжья нотация (CamelCase)**. В ней каждое слово начинается с большой буквы, между словами нет разделителей.

ClassArchers, GameEngine

➡ Использования знака подчеркивания в качестве первого символа идентификатора означает, что данный идентификатор является частным (закрытым от использования вне класса).

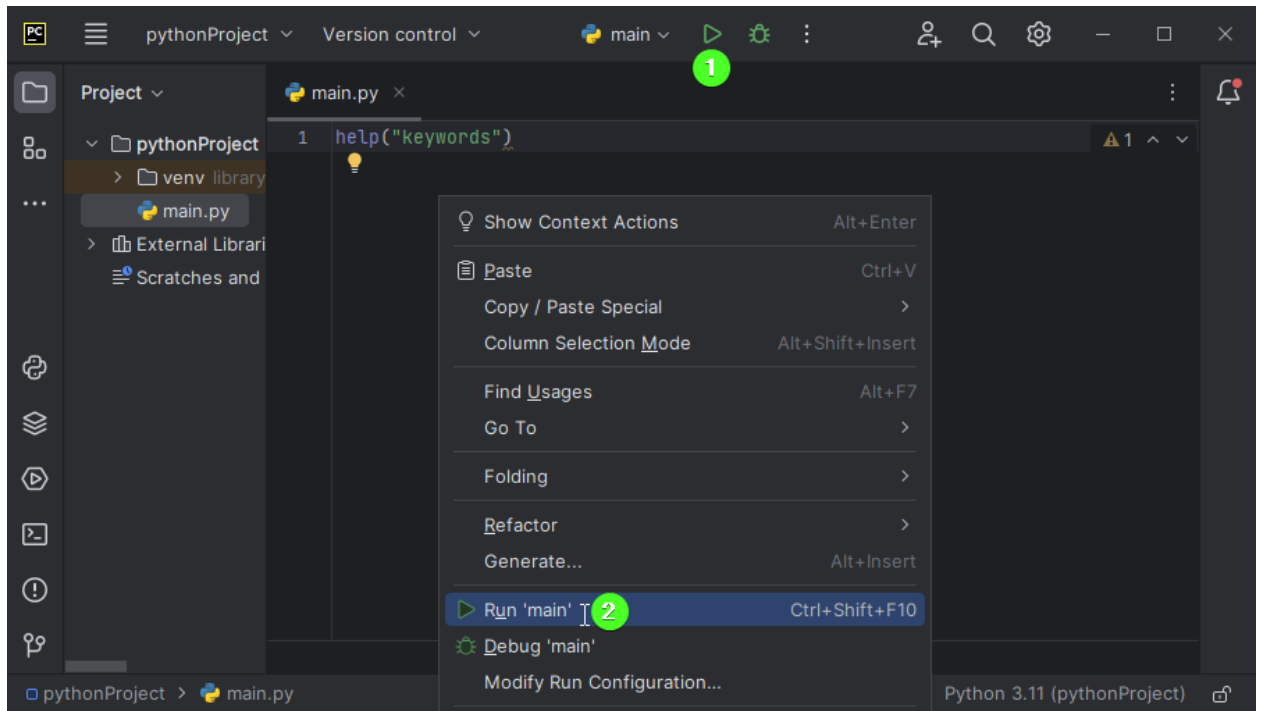
➡ Если идентификатор начинается и заканчивается двумя знаками подчеркивания (например, `__init__`), это означает, что он является специальным именем, определенным внутри языка.

Также есть зарезервированные слова, которые нельзя использовать в качестве имени переменной или любого другого идентификатора.

Постарайтесь запомнить эти слова, а лучше выпишите их в блокнот или тетрадь.

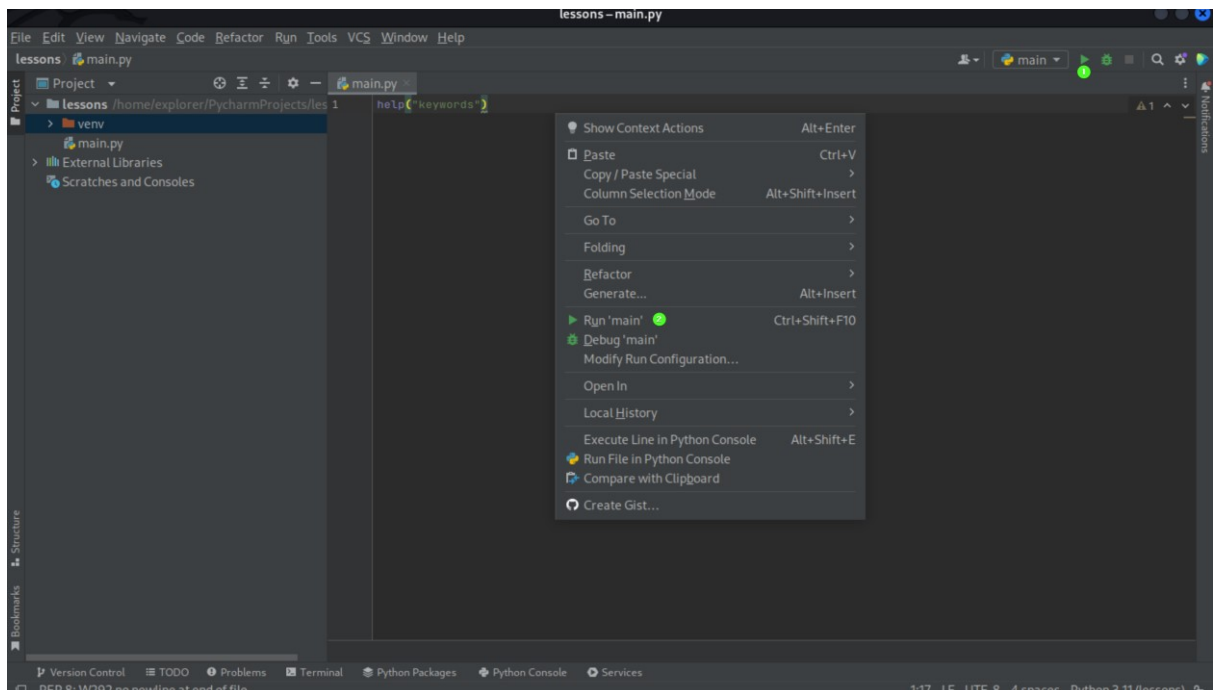
Вывести перечень этих слов можно, добавив в наш файл `test.py` инструкцию `help("keywords")`. `help` – это имя функции, которая выводит справку по объекту, который мы передаём в скобках. В данном случае мы выводим на экран справку по ключевым словам – `keywords`. Так как это строка, нужно заключить её в кавчки.

Таким же образом мы можем получить справку по любому объекту в python, передав его имя в скобки функции `help`.

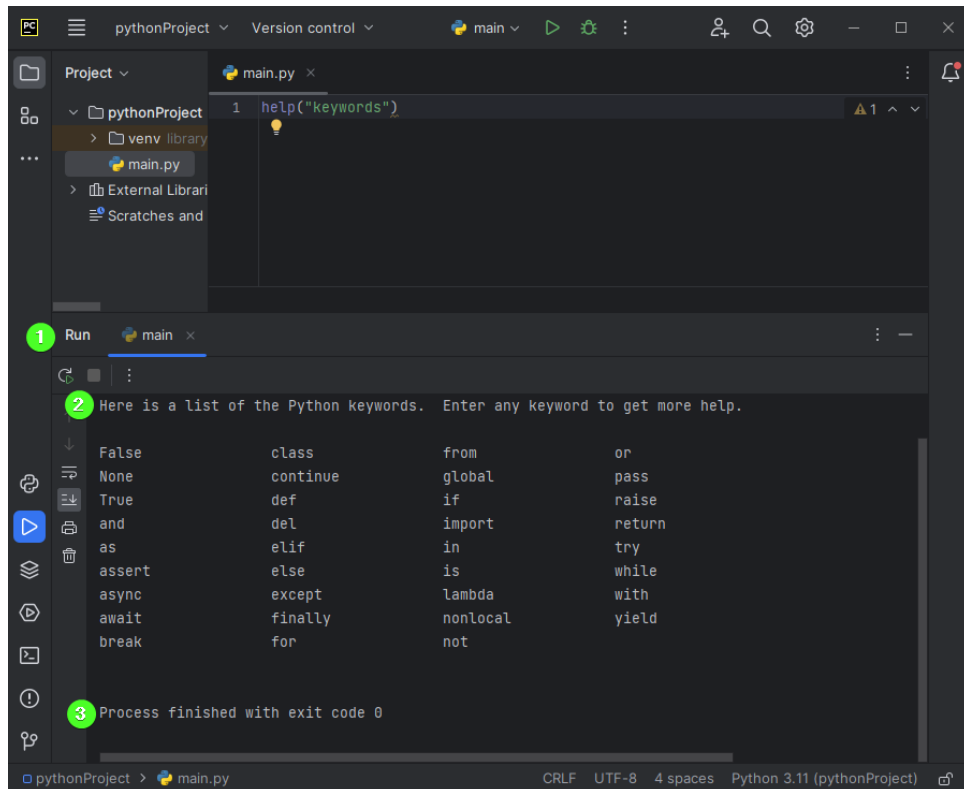


Выполнить код написанной программы можно несколькими способами. Самый простой – нажать на зелёную стрелку (1) в верхней части экрана, так же можно кликнуть в поле вашего кода правой кнопкой мыши и выбрать в выпадающем списке пункт Run 'main' (2)

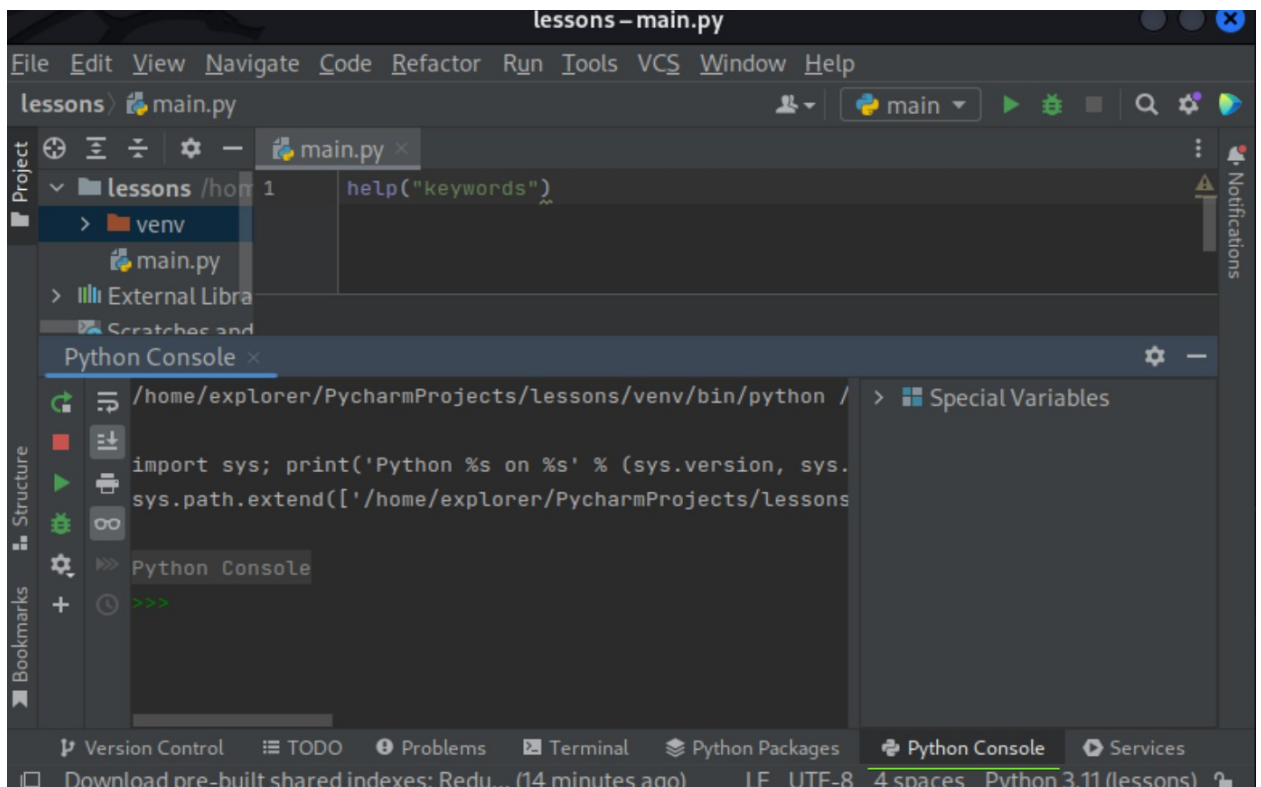
В Linux оформление IDE будет немного отличаться, но символ зелёной стрелки подскажет вам, как запустить код:

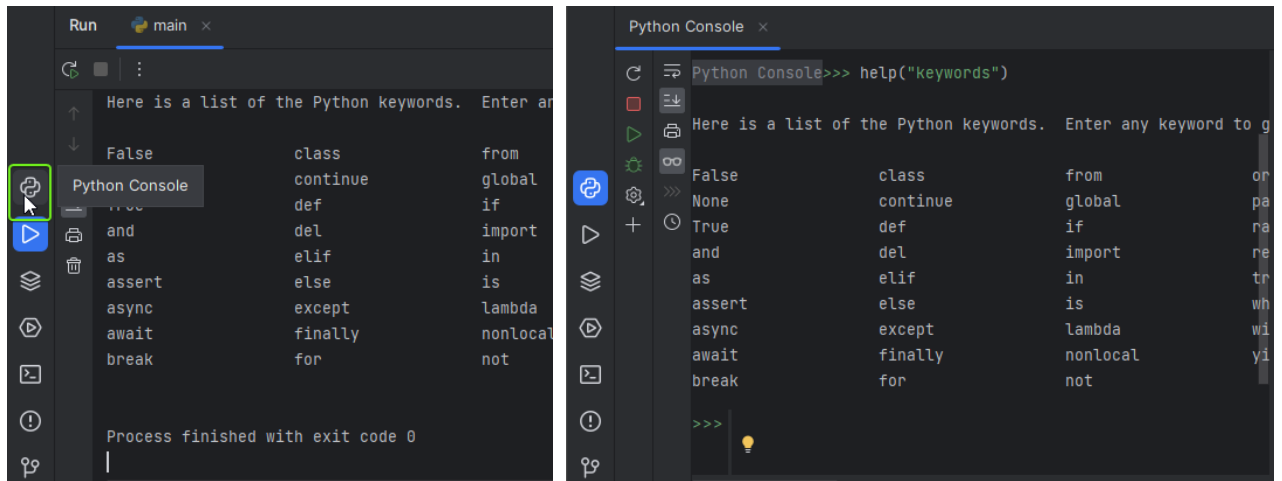


Старайтесь запоминать и горячие клавиши – Ctrl+Shift+F10 выполнит запуск кода. После выполнения кода программы PyCharm откроет окно (1) Run и выведет там (2) результат работы кода и (3) строку с информацией о завершении программы. В данном примере программа завершилась штатно, значит, мы получили код возврата 0:



Мы можем, не покидая среды разработки, открыть интерактивный режим python, нажав на вкладку **Python Console**.

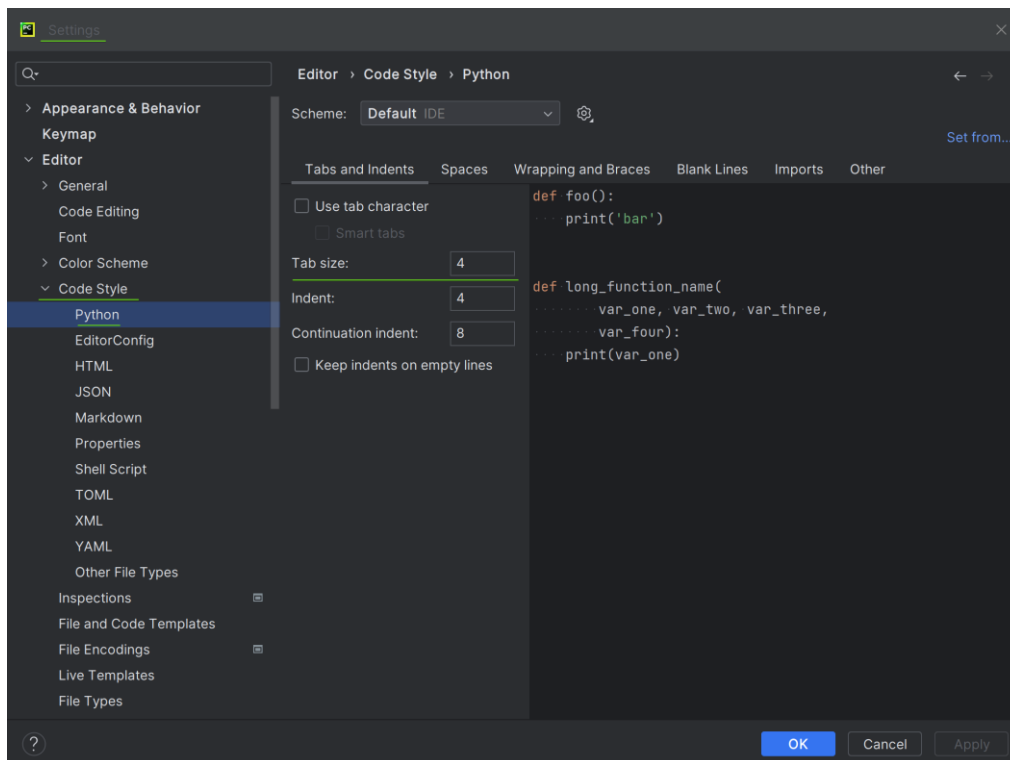




Этот режим полезен в тех случаях, когда мы хотим проверить выполнение какой-то части кода, найти справку по функции или объекту, не создавая новый файл.

Строки и отступы

Python читает код построчно сверху вниз, а конец строки является концом инструкции. Вложенный код в большинстве языков программирования обычно размещается в фигурных скобках {}. В **python** же вложенный код отделяется от основного кода отступами. По умолчанию отступ равен 4 пробелам. Если вдруг у вас изменился этот отступ, найти параметр, отвечающий за количество пробелов, можно в настройках **Editor > Code Style > Python > Tab size**:



Также вложенный код от основного кода отделяется двоеточием, после которого идёт перенос строки. Пример:

```
main.py x
1 var_1 = "Codeby" # Объявление переменной var_1
2 if var_1 == "Codeby": # Условный оператор (будет рассмотрен в следующих уроках)
3     print("Hello Codeby!") # Вложенный код вывод строки, в случае выполнения условия (отступ 4 пробела)
4 else: # условный оператор ветвления
5     print("Hello Wolrd!") # Вложенный код вывод строки, в случае выполнения условия (отступ 4 пробела)
6
```

Точка с запятой (;) позволяет вводить несколько инструкций на одной строке. Но такой подход снижает читаемость кода, и считается непитоничным. Поэтому лучше избегать такого написания кода.

```
>>> name = 'Vanya'; surname = "Petrov"; print(name, surname)
Vanya Petrov
```

Обратите внимание – на скриншоте выше в интерактивном режиме вывод сделан через явное указание функции `print`. На самом деле, в этом режиме писать `print` не нужно, интерпретатор и так всё поймёт. Поэтому достаточно только написать имена переменных, чтобы получить их значения.

```
>>> name = "Vanya"
>>> surname = "Petrov"
>>> name, surname
('Vanya', 'Petrov')
```

В Python можно использовать одинарные ('), двойные ("), и тройные (""" или """) кавычки, чтобы обозначить строчный тип данных, при этом начинаться и заканчиваться строка должна одинаковыми кавычками. Какие кавычки использовать - одинарные или двойные, не имеет значения. Строка, занимающая несколько строк кода должна быть обрамлена тройными кавычками.

Пример кавычек, и встроенных комментариев, располагающихся за строкой:

```
test.py x
1 name = 'Vanya' # одинарные кавычки
2 nickname = "Vano" # двойные кавычки
3 description = '''Bla-bla-bla
4 long text here''' # тройные кавычки
5
```

Обратите внимание – когда вы используете тройные кавычки, оформление – форматирование строки сохраняется, то есть переносы, добавленные к строке, будут отображены и при печати переменной с такой строкой:

```

1 description = '''Bla-bla-bla
2 long text here''' # тройные кавычки
3
4 print(description)

```

Run: main ×

```

/home/explorer/PycharmProjects/lesson
Bla-bla-bla
long text here

```

Как видите, содержимое вывелось в таком же виде, как и было создано в переменной. Если бы кавычки были двойными или одинарными, текст распечатался бы в одну строку.

```

1 description = 'Bla-bla-bla \
2 long text here' # тройные кавычки
3
4 print(description)

```

Run: main ×

```

/home/explorer/PycharmProjects/lessons/ve
Bla-bla-bla long text here

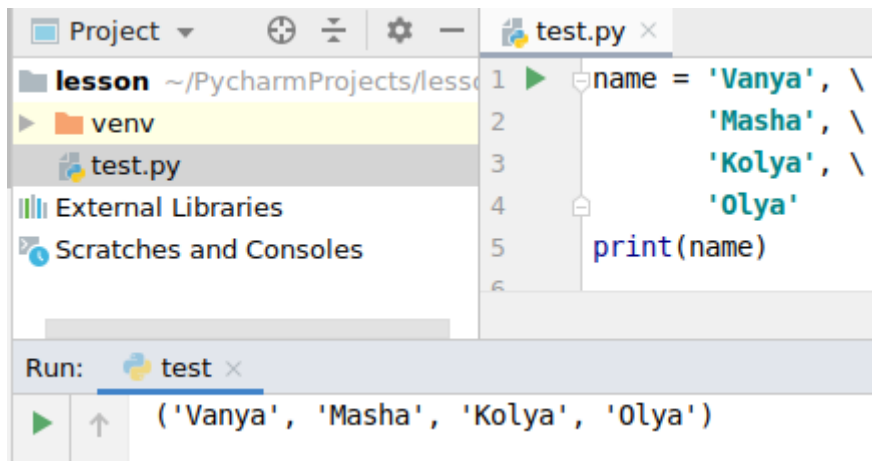
```

Комментарии в Python

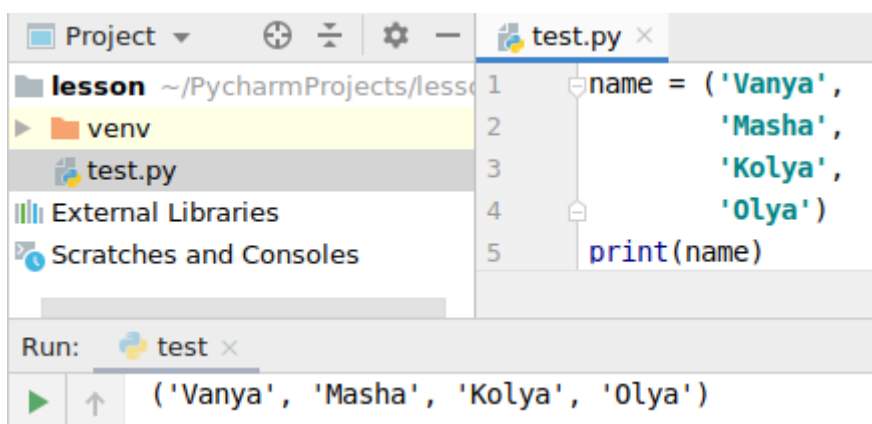
Символ решетки (#) в Python обозначает начало комментария. Все символы после # и до конца строки являются частью комментария, и **Python** игнорирует их при чтении кода. Если комментарий встро́чный, то есть находится на той же строке, которую описывает, то отделяться от кода он должен двумя пробелами, далее идёт символ решётки, потом ещё один пробел и дальше текст комментария.

Комментарии также можно размещать выше строки кода на новой строке. Чтобы не превышать допустимую стандартом PEP8 длину строки кода в 80 символов, старайтесь использовать как раз такие комментарии.

Как показано в примере выше - символ переноса строки (\) – обратный слеш, позволяет при необходимости визуально разбивать длинные строки.



Выражения, которые находятся внутри скобок: квадратных [], фигурных { } или круглых (), не нуждаются в символе переноса строки.



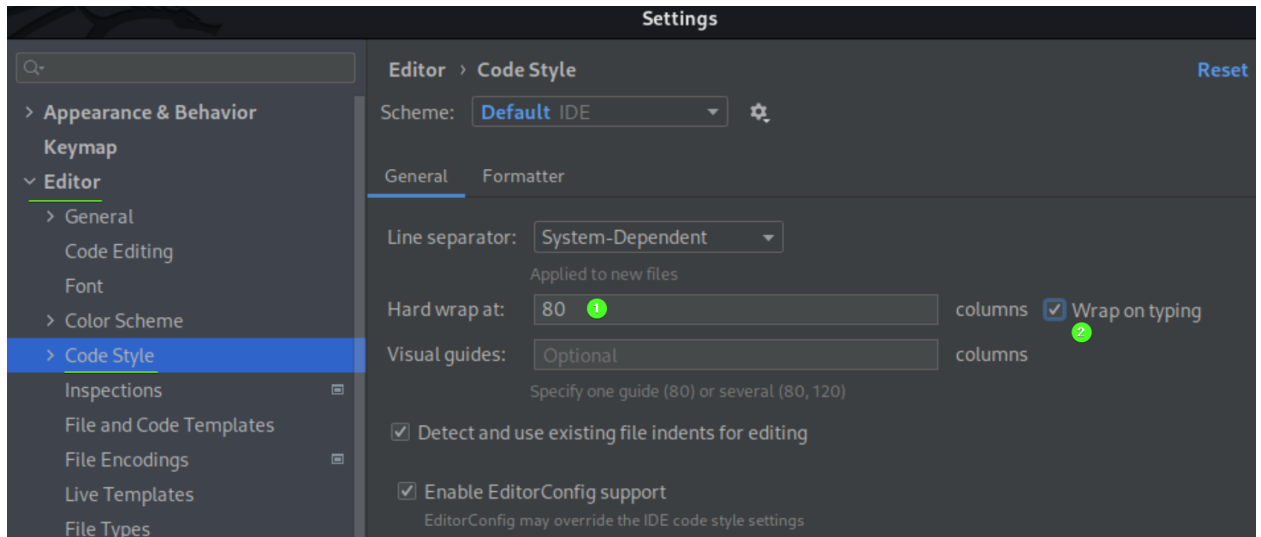
Настройки IDE PyCharm

Установка ограничения максимальной длины строки

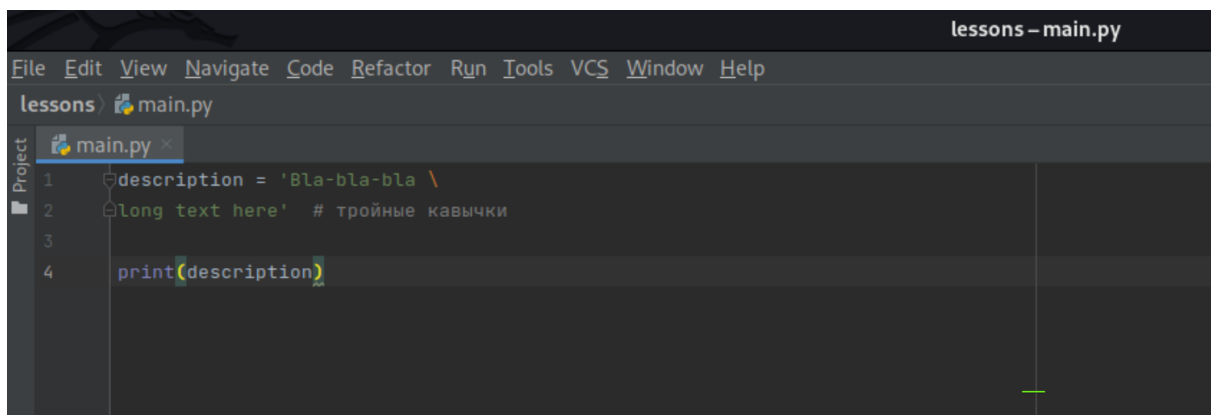
Для того, чтобы сделать код `python` всех программистов единообразным, были разработаны стандарты, которым мы так же будем следовать. Главным стандартом, регламентирующим оформление кода, является [PEP8](#). Среди прочего в нём предлагается ограничивать длину строки кода в 80 символов.

Чтобы каждый раз не подсчитывать количество символов, превышающих допустимую длину, PyCharm может нарисовать вертикальную черту на нужной длине. Для этого в настройках IDE перейдите в **Editor > Code Style > Hard wrap at** в поле справа укажите цифру 80.

Вы так же можете поставить галочку в блоке **Wrap on typing**, чтобы строка автоматически переносилась на новую в момент, когда, печатая, вы превышаете указанный слева диапазон.



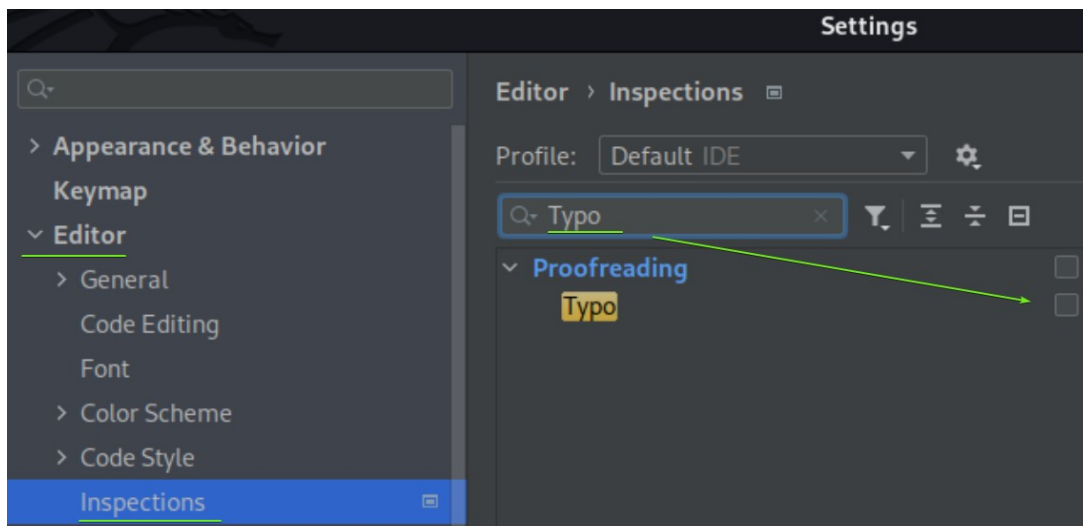
После этого вы увидите вертикальную линию на позиции 80 символа.



Отключение проверки орфографии

По умолчанию PyCharm проверяет не только синтаксис кода, но и наличие орфографических ошибок, но делает это только на английском языке. Нам подобная информация – волнистое подчёркивание – будет только мешать. Так что мы смело отключаем проверку орфографии.

Переходим в Настройки **Editor > Inspections** и в поле с лупой введите **Typo**. Самую нижнюю галочку в списке нужно снять и сохранить настройки.



Домашнее задание:

1. Проверьте установленные версии Python, и установите разные версии, например 2.x, 3.9.x, 3.10.x
2. Создайте алиасы для разных версий - python2, python3.11
3. Установите IDE PyCharm и создайте для него алиас PyCharm.
4. Разберитесь со скриптовым и интерактивным режимом – попробуйте вывести в обоих режимах слова/математические выражения.
5. Получите справку по функции print и по функции input
6. Зарегистрируйтесь на форуме [Codeby](#) – именно там мы будем отправлять домашние задания
7. Ознакомьтесь с [инструкцией](#) по выполнению домашнего задания
8. Выполнить тест, отправить ответы на форуме, указав ваш ник в таблице или логин почты, с которой вы регистрировались в школе.